

Informe de Laboratorio 05

Tema: Base de Datos

Nota

Estudiantes	Escuela	Asignatura
Gustavo Alonso Carrillo Villalta Fredy Jos� Arag�n Carpio Diego Marcelo Arce Coaquira Jos� Manuel Bravo Rojas	Escuela Profesional de Ingenier�a de Sistemas	Desarrollo de Aplicaciones Web Semestre: I C�digo: 2025004 Grupo: Desarrolladores

Laboratorio	Tema	Duraci�n
05	Base de Datos	04 horas

Semestre acad�mico	Fecha de inicio	Fecha de entrega
2026 - A	05 de Mayo de 2026	12 de Mayo de 2026

1. Laboratorio 05: Base de datos

En este laboratorio se elabor  el modelo l gico DER y su implementaci n f sica en PostgreSQL, siguiendo est ndares de nomenclatura en ingl s, con tablas en plural y camelCase, incluyendo los atributos obligatorios **id**, **status**, **created**, **modified**, **created_id** y **modified_id**.

El modelo fue implementado en **Supabase**, plataforma de Backend como Servicio (BaaS) disponible en la nube, la cual ofrece una base de datos PostgreSQL gestionada, autenticaci n y API REST de forma integrada.

Las relaciones N:M fueron representadas mediante tablas intermedias ordenadas alfab ticamente, y los atributos de relaci n siguen la convenci n **tabla_id** para garantizar consistencia y legibilidad del esquema.

Para la documentaci n, se elabor  un archivo **README.md** en el repositorio de GitHub que detalla el modelo DER, el modelo f sico, la implementaci n en Supabase y los anexos correspondientes.

1.1. Descripci n y Objetivos

En este laboratorio se elabor  el modelo l gico DER y su implementaci n f sica en PostgreSQL para una base de datos relacional, siguiendo est ndares de nomenclatura en ingl s con tablas en plural y camelCase. Todas las tablas incluyen los atributos obligatorios **id**, **status**, **created**, **modified**, **created_id** y **modified_id**, garantizando trazabilidad y consistencia en el esquema.

La base de datos fue implementada en **Supabase**, plataforma de Backend como Servicio (BaaS) disponible en la nube, que proporciona una instancia PostgreSQL gestionada junto con autenticación y API REST integradas. Las relaciones N:M fueron representadas mediante tablas intermedias ordenadas alfabéticamente, y los atributos de relación siguen la convención `tabla_id`.

Como parte de las buenas prácticas, se elaboró un archivo `README.md` en el repositorio de GitHub que documenta el modelo DER, el modelo físico, la implementación en Supabase y los anexos correspondientes.

- Elaborar el modelo lógico DER que represente correctamente las entidades, atributos y relaciones del sistema.
- Implementar el modelo físico en PostgreSQL aplicando estándares de nomenclatura en inglés, plural y camelCase.
- Incluir en todas las tablas los atributos obligatorios `id`, `status`, `created`, `modified`, `created_id` y `modified_id`.
- Nombrar los atributos de relación siguiendo la convención `tabla_id` para garantizar consistencia en el esquema.
- Ordenar alfabéticamente las tablas intermedias producto de relaciones N:M.
- Implementar la base de datos en **Supabase** como plataforma de Backend como Servicio (BaaS) en la nube.
- Documentar toda la práctica mediante un archivo `README.md` en el repositorio de GitHub.

1.2. Resumen del Ejercicio

En este ejercicio se elaboró el modelo lógico DER y su implementación física en PostgreSQL para una base de datos relacional de una cafetería, siguiendo estándares de nomenclatura en inglés con tablas en plural y camelCase. Todas las tablas incluyen los atributos obligatorios `id`, `status`, `created`, `modified`, `created_id` y `modified_id`.

La base de datos fue implementada en **Supabase**, plataforma de Backend como Servicio (BaaS) disponible en la nube, que proporciona una instancia PostgreSQL gestionada junto con autenticación y API REST integradas.

Finalmente, se documentó todo el proceso en un archivo `README.md` en el repositorio de GitHub, incluyendo el modelo DER, los scripts SQL y las capturas de la implementación en Supabase.

1.3. Entregables

El proyecto se documenta y entrega a través de:

- **URL del Repositorio:** <https://github.com/FredyAragon/Cafe-Sys/tree/main>
- **Informe:** <https://github.com/FredyAragon/Cafe-Sys/blob/main/Informes/Informe-Lab5-DAW.pdf>

1.4. Entorno de Implementación

Para la implementación de la base de datos se utilizaron las siguientes herramientas: PostgreSQL como motor de base de datos relacional, Supabase como plataforma BaaS en la nube, y el editor de consultas SQL integrado en Supabase para la ejecución de los scripts.

1.4.1. Acceso a Supabase

Se accede al panel de Supabase mediante:

- **Panel Supabase:** <https://supabase.com/dashboard>
- **Editor SQL:** Project → SQL Editor → New Query

1.4.2. Ejecución de scripts SQL

Los scripts se ejecutan directamente en el editor SQL de Supabase en el siguiente orden:

1. 01_create_tables.sql -- Creación de tablas
2. 02_insert_data.sql -- Inserción de datos de prueba

1.4.3. Verificación de tablas

Una vez ejecutados los scripts, las tablas pueden verificarse desde:

Supabase Dashboard Table Editor Ver tablas creadas

O mediante la siguiente consulta SQL:

```
SELECT table_name
FROM information_schema.tables
WHERE table_schema = 'public'
ORDER BY table_name;
```

2. Equipos, materiales y temas utilizados

- Sistema Operativo: Windows 11
- PostgreSQL como motor de base de datos relacional
- Supabase como plataforma BaaS en la nube
- Editor SQL integrado en Supabase
- Visual Studio Code
- Git y GitHub
- dbdiagram.io para la elaboración del modelo DER
- Draw.io / Lucidchart para diagramas adicionales

3. URL del Repositorio en GitHub

El ejercicio desarrollado en el presente laboratorio se encuentra disponible en el repositorio oficial alojado en GitHub, perteneciente a CafeSys. A continuación, se presenta el enlace al repositorio y las rutas de acceso al proyecto:

- **URL del Repositorio:** <https://github.com/FredyAragon/Cafe-Sys.git>
- **Scripts SQL:** <https://github.com/FredyAragon/Cafe-Sys/blob/main/BD/bd.sql>

4. Desarrollo de actividades del laboratorio

4.1. Estructura del Proyecto

El proyecto está organizado de la siguiente manera:

BD/

```
|----README.md
|----bd.sql
|----.gitignore

|----Informes/
|      +----Informe-Lab5-DAW.pdf

|----backend/
|      |----api/
|      |----config/
|      |----manage.py
|      +----requirements.txt

+----webapp/
|      +----__pycache__/
|      |----controllers.cpython-312.pyc
|      +----databases.cpython-312.pyc
```

4.2. Resoluci n del Ejercicio

4.2.1. Arquitectura de la soluci n

La base de datos fue dise ada siguiendo el modelo relacional, estructurando las entidades en tablas normalizadas con sus respectivas claves primarias, for neas y restricciones. El dise o se organiz  en cuatro versiones incrementales, partiendo de un n cleo base de 11 tablas hasta alcanzar 22 tablas con m dulos de env os, ubigeo y recetas.

4.2.2. L gica del modelo

El flujo del dise o se describe de la siguiente manera:

- Se identificaron las entidades principales del sistema (usuarios, productos, pedidos) y sus relaciones.
- Se elabor  el modelo l gico DER representando entidades, atributos y relaciones N:M, 1:N y 1:1.
- Se implement  el modelo f sico en PostgreSQL aplicando las convenciones de nomenclatura requeridas.
- Las tablas intermedias producto de relaciones N:M fueron nombradas en orden alfab tico.
- Se insertaron registros de prueba para validar la integridad referencial del esquema.
- Finalmente, se desple  la base de datos en Supabase y se verific  su correcta implementaci n.

4.2.3. Organizaci n del proyecto

La estructura del proyecto sigue una separaci n por responsabilidades:

- **sql/**: Contiene los scripts de creaci n e inserci n.
 - **01.create.tables.sql**: Define todas las tablas con sus restricciones y claves for neas.
 - **02.insert.data.sql**: Inserta los registros de prueba en cada tabla.
- **img/**: Contiene el diagrama DER y capturas de Supabase.
- **README.md**: Documenta el modelo, los scripts y el proceso de implementaci n.

5. An lisis del Modelo

5.1. Modelo Lógico DER

El modelo entidad-relación (DER) representa la estructura conceptual de la base de datos del sistema de cafetería. En él se identifican 12 entidades principales, incluyendo usuarios, roles, productos, categorías, pedidos, detalles de pedidos, promociones e inventario, así como sus relaciones de tipo 1:N y N:M.

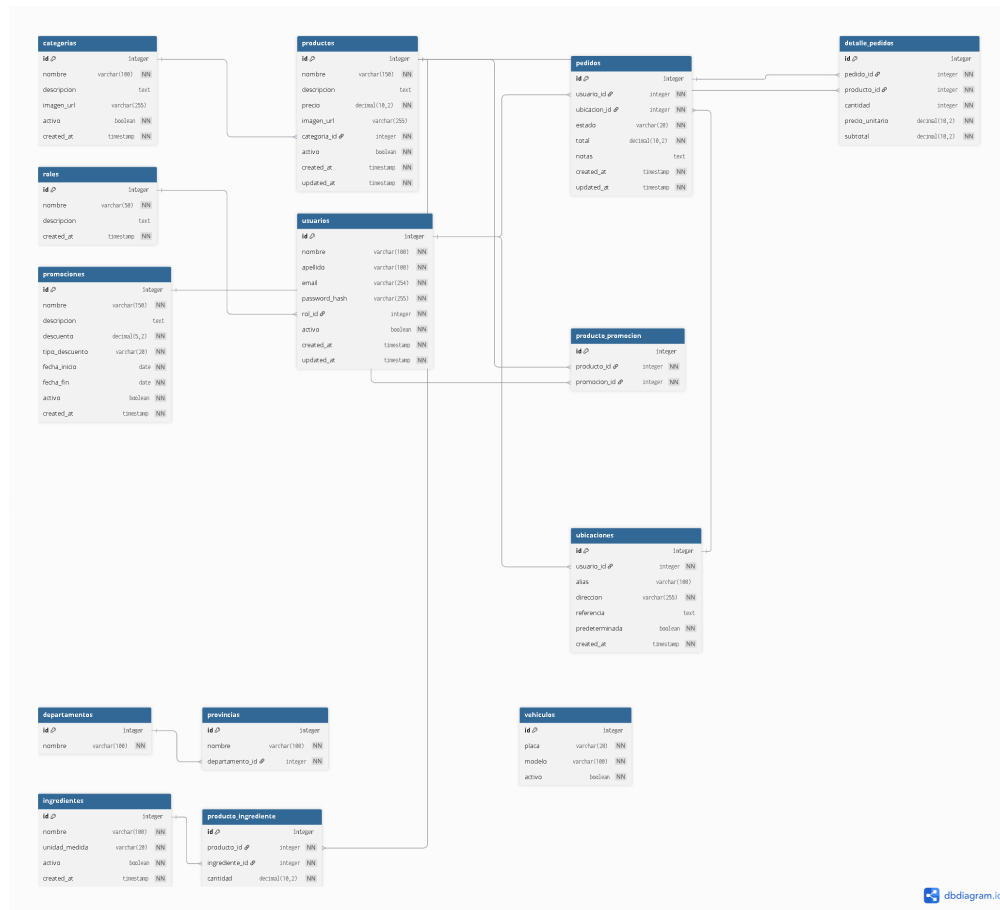


Figura 1: Modelo lógico DER del sistema de cafetería con sus 12 tablas y relaciones.

5.2. Modelo Físico — Scripts SQL

A partir del modelo lógico se implementó el modelo físico en PostgreSQL, siguiendo las convenciones de nomenclatura establecidas para tablas, atributos y claves foráneas.

5.2.1. Tabla roles

Listing 1: Creación de la tabla roles.

```
CREATE TABLE roles (
    id SERIAL PRIMARY KEY,
    nombre VARCHAR(50) NOT NULL UNIQUE,
    descripcion TEXT,
    status BOOLEAN NOT NULL DEFAULT TRUE,
    created TIMESTAMP NOT NULL DEFAULT NOW(),
    modified TIMESTAMP NOT NULL DEFAULT NOW(),
```

```
        created_id INTEGER,  
        modified_id INTEGER  
    );
```

5.2.2. Tabla usuarios

Listing 2: Creación de la tabla usuarios.

```
CREATE TABLE usuarios (  
    id          SERIAL PRIMARY KEY,  
    nombre      VARCHAR(100) NOT NULL,  
    apellido    VARCHAR(100) NOT NULL,  
    email       VARCHAR(150) NOT NULL UNIQUE,  
    passwordHash VARCHAR(255) NOT NULL,  
    rolId       INTEGER NOT NULL REFERENCES roles(id),  
    status      BOOLEAN NOT NULL DEFAULT TRUE,  
    created     TIMESTAMP NOT NULL DEFAULT NOW(),  
    modified    TIMESTAMP NOT NULL DEFAULT NOW(),  
    created_id  INTEGER,  
    modified_id INTEGER  
);
```

5.2.3. Tabla productos

Listing 3: Creación de la tabla productos.

```
CREATE TABLE productos (  
    id          SERIAL PRIMARY KEY,  
    nombre      VARCHAR(150) NOT NULL,  
    descripcion TEXT,  
    precio      NUMERIC(10,2) NOT NULL CHECK (precio > 0),  
    imageUrl    VARCHAR(255),  
    categoriaId INTEGER NOT NULL REFERENCES categorias(id),  
    status      BOOLEAN NOT NULL DEFAULT TRUE,  
    created     TIMESTAMP NOT NULL DEFAULT NOW(),  
    modified    TIMESTAMP NOT NULL DEFAULT NOW(),  
    created_id  INTEGER,  
    modified_id INTEGER  
);
```

5.2.4. Tabla pedidos

Listing 4: Creación de la tabla pedidos.

```
CREATE TABLE pedidos (  
    id          SERIAL PRIMARY KEY,  
    usuarioId   INTEGER NOT NULL REFERENCES usuarios(id),  
    ubicacionId INTEGER NOT NULL REFERENCES ubicaciones(id),  
    estado      VARCHAR(20) NOT NULL CHECK (  
        estado IN (  
            'pendiente',  
            'en_preparacion',  
            'listo',  
            'entregado',  
        )  
    );
```

```
        'cancelado'
    )
),
total      NUMERIC(10,2) NOT NULL CHECK (total >= 0),
notas      TEXT,
status     BOOLEAN NOT NULL DEFAULT TRUE,
created    TIMESTAMP NOT NULL DEFAULT NOW(),
modified   TIMESTAMP NOT NULL DEFAULT NOW(),
created_id INTEGER,
modified_id INTEGER
);
```

5.2.5. Tabla detalle_pedidos

Listing 5: Creación de la tabla detalle_*pedidos*.

```
CREATE TABLE detalle_pedidos (
    id          SERIAL PRIMARY KEY,
    pedidoId    INTEGER NOT NULL REFERENCES pedidos(id),
    productoId  INTEGER NOT NULL REFERENCES productos(id),
    cantidad    INTEGER NOT NULL CHECK (cantidad > 0),
    precioUnitario NUMERIC(10,2) NOT NULL,
    subtotal    NUMERIC(10,2) NOT NULL,
    status      BOOLEAN NOT NULL DEFAULT TRUE,
    created     TIMESTAMP NOT NULL DEFAULT NOW(),
    modified    TIMESTAMP NOT NULL DEFAULT NOW(),
    created_id  INTEGER,
    modified_id INTEGER,
    UNIQUE (pedidoId, productoId)
);
```

5.3. Despliegue y Ejecución del Proyecto

El sistema fue desarrollado con Django como framework backend y PostgreSQL como motor de base de datos. Para ejecutar el proyecto es necesario contar con Python 3.10 o superior, PostgreSQL en ejecución y la base de datos previamente creada y poblada con el script `bd.sql`.

Nota importante: Django no crea automáticamente la base de datos; únicamente se conecta a una base de datos PostgreSQL ya existente.

5.3.1. Requisitos previos

- Python 3.10 o superior.
- PostgreSQL instalado y en ejecución.
- Base de datos del proyecto creada y cargada.

5.3.2. Pasos para ejecutar el sistema

Listing 6: Comandos para levantar el proyecto localmente.

```
git clone https://github.com/FredyAragon/Cafe-Sys.git
cd Cafe-Sys/backend
python -m venv venv
venv\Scripts\activate
pip install -r requirements.txt
```

```
python manage.py runserver 8081
```

Una vez ejecutados los comandos anteriores, el servidor estará disponible en la siguiente dirección:

`http://localhost:8081`

5.3.3. Acceso al panel de administración

Para crear un usuario administrador se ejecuta el siguiente comando:

Listing 7: Creación de un superusuario en Django.

```
python manage.py createsuperuser
```

Posteriormente se puede acceder al panel administrativo desde:

`http://localhost:8081/admin`

Desde esta interfaz es posible gestionar productos, usuarios, pedidos, promociones e inventario mediante una interfaz gráfica provista por Django Admin.

6. Resultados

A continuación se presentan evidencias de la implementación final de la base de datos en PostgreSQL y su correcta integración con el backend desarrollado en Django.

✓ **Tablas (30)**

- >  auth_group
- >  auth_group_permissions
- >  auth_permission
- >  auth_user
- >  auth_user_groups
- >  auth_user_user_permissions
- >  carrito
- >  carrito_detalle
- >  categorias
- >  detalle_pedidos
- >  django_admin_log
- >  django_content_type
- >  django_migrations
- >  django_session
- >  envios
- >  ingredientes
- >  inventario
- >  inventario_ingredientes

▼		productos
▼		Columnas (9)
		id
		nombre
		descripcion
		precio
		imagen_url
		categoria_id
		activo
		created_at
		updated_at

Figura 3: Estructura de la tabla `productos`, mostrando los tipos de datos definidos para cada atributo.

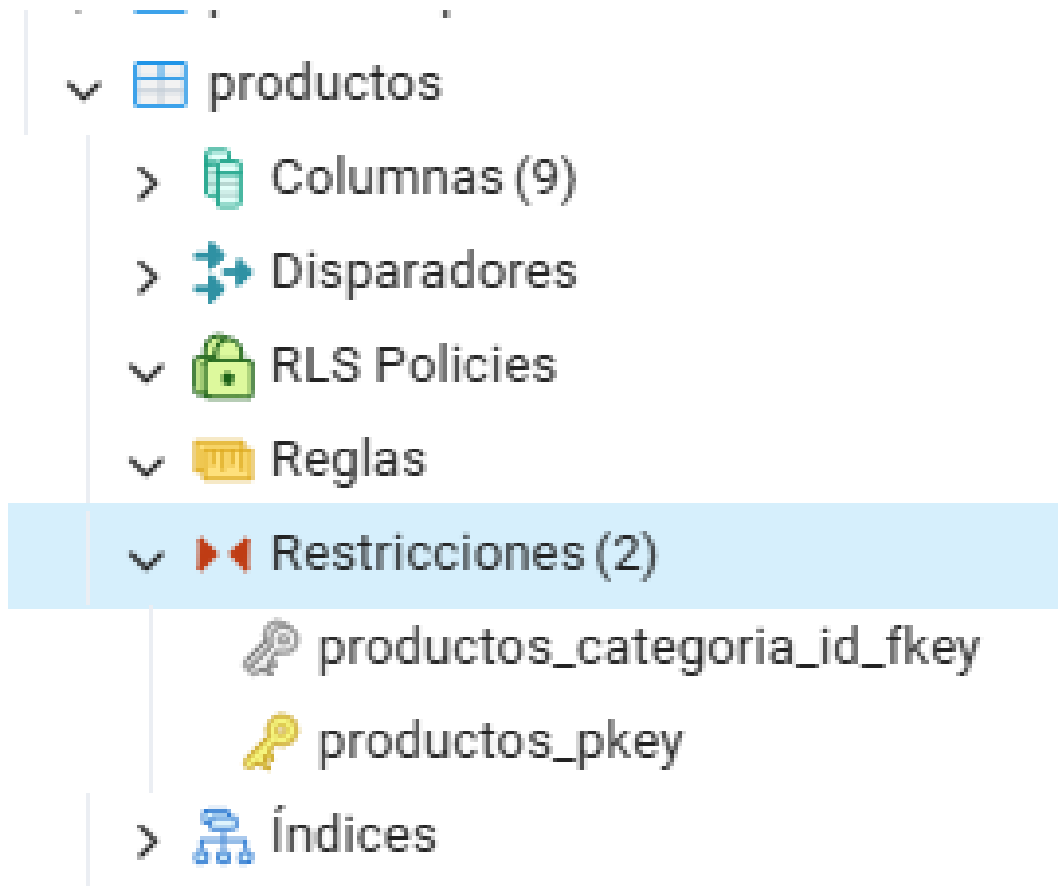


Figura 4: Restricciones de clave foránea (Foreign Keys) definidas en la tabla `productos`.

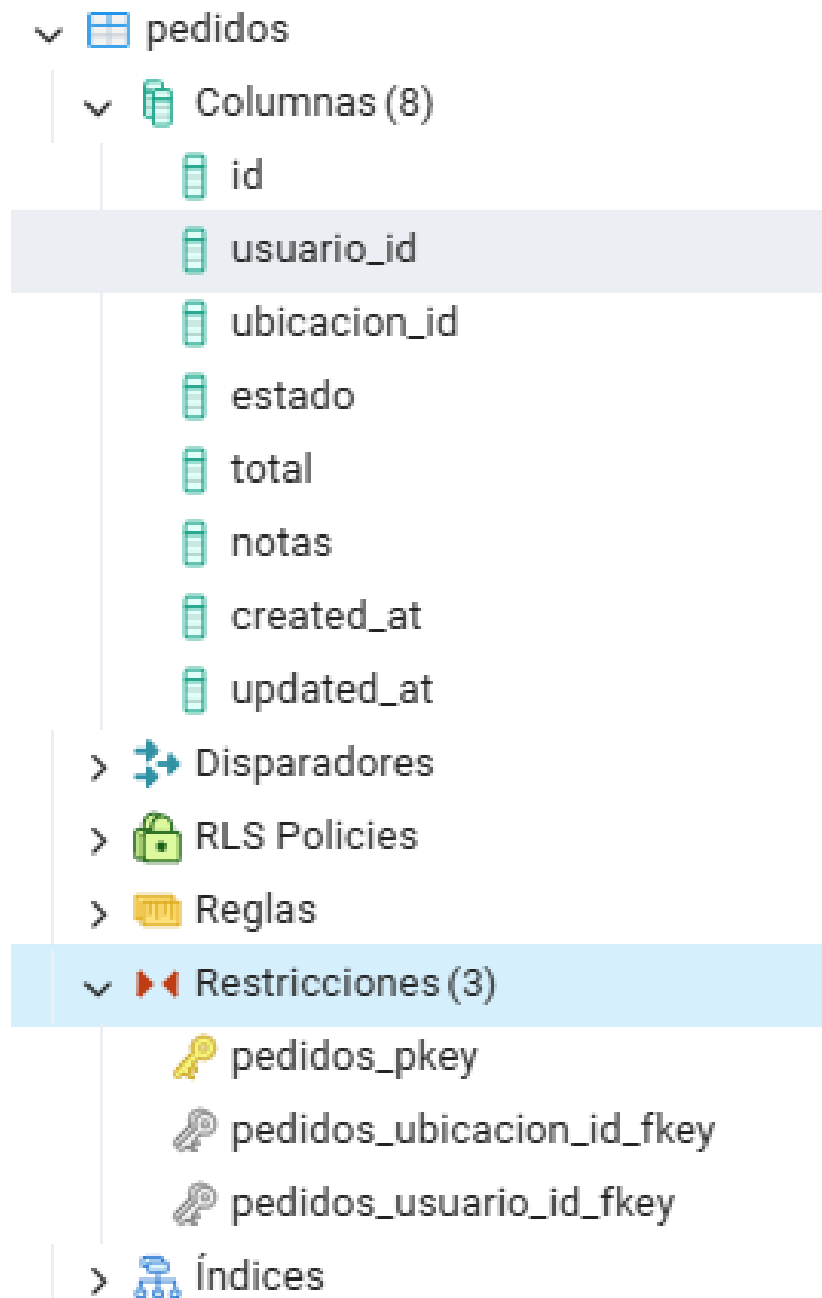


Figura 5: Estructura de la tabla `pedidos` evidenciando su dise o relacional, adicional se muestra las claves for neas de la tabla `pedidos`, mostrando su relaci n con otras entidades del sistema

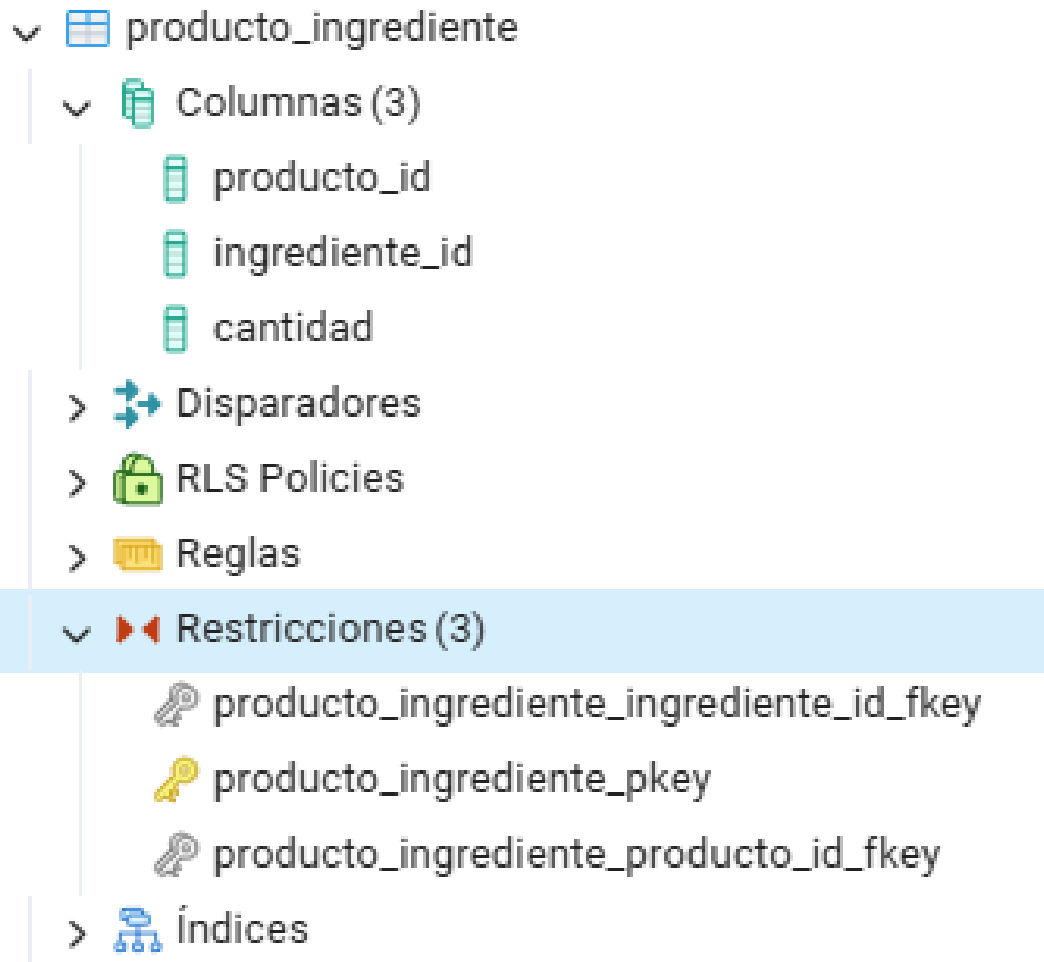


Figura 6: Tabla intermedia **producto_ingrediente** que resuelve la relación muchos a muchos entre productos e ingredientes.

Data Output Mensajes Notificaciones					
	id [PK] integer	nombre character varying (150)	precio numeric (10,2)	categoria_id integer	activo boolean
1	11	Espresso	6.50	1	true
2	12	Americano	7.00	1	true
3	13	Cappuccino	9.50	1	true
4	14	Latte	9.00	1	true
5	15	Mocha	10.00	1	true
6	16	Macchiato	8.00	1	true
7	17	Flat White	9.00	1	true
8	18	Affogato	11.00	1	true
9	19	Cold Brew	8.50	1	true
10	20	Frapp�	9.80	1	true

Figura 7: Registros de prueba insertados en la tabla **productos** verificando el correcto almacenamiento de datos

	id [PK] integer	nombre character varying (100)
1	1	Caf�s Calientes
2	2	Caf�s Fr�os
3	3	Postres

Figura 8: Registros almacenados en la tabla **categorias**, comprobando la integridad referencial

Consulta Historial de Consultas [Scratch Pad](#)

```
1 SELECT * FROM productos;
```

Data Output Mensajes Notificaciones

Showing rows: 1 to 10 Page No: 1 of 1

	id [PK] integer	nombre character varying (150)	descripcion text	precio numeric (10,2)	imagen_url character varying (255)	categoria_id integer	activo boolean
1	11	Espresso	Café concentrado preparado a alta presión, sabor intenso y aroma fuerte.	6.50	https://example.com/img/espresso.jpg	1	true
2	12	Americano	Espresso diluido en agua caliente, sabor suave pero aromático.	7.00	https://example.com/img/americano.jpg	1	true
3	13	Cappuccino	Espresso con leche vaporizada y espuma cremosa.	9.50	https://example.com/img/cappuccino.jpg	1	true
4	14	Latte	Café espresso con abundante leche caliente y ligera espuma.	9.00	https://example.com/img/latte.jpg	1	true
5	15	Mocha	Café espresso con chocolate, leche vaporizada y crema.	10.00	https://example.com/img/mocha.jpg	1	true
6	16	Macchiato	Espresso manchado con una pequeña cantidad de leche.	8.00	https://example.com/img/macchiato.jpg	1	true
7	17	Fiat White	Café espresso con microespuma de leche, textura sedosa.	9.00	https://example.com/img/flatwhite.jpg	1	true
8	18	Affogato	Helado de vainilla bañado con un espresso caliente.	11.00	https://example.com/img/affogato.jpg	1	true
9	19	Cold Brew	Café infusionado en frío durante 12 horas, sabor suave.	8.50	https://example.com/img/coldbrew.jpg	1	true
10	20	Frappé	Café frío batido con hielo, leche y azúcar.	9.80	https://example.com/img/frappe.jpg	1	true

Figura 9: Ejecución de una consulta **SELECT** desde el Query Tool de PGAdmin demostrando el acceso correcto a los datos.

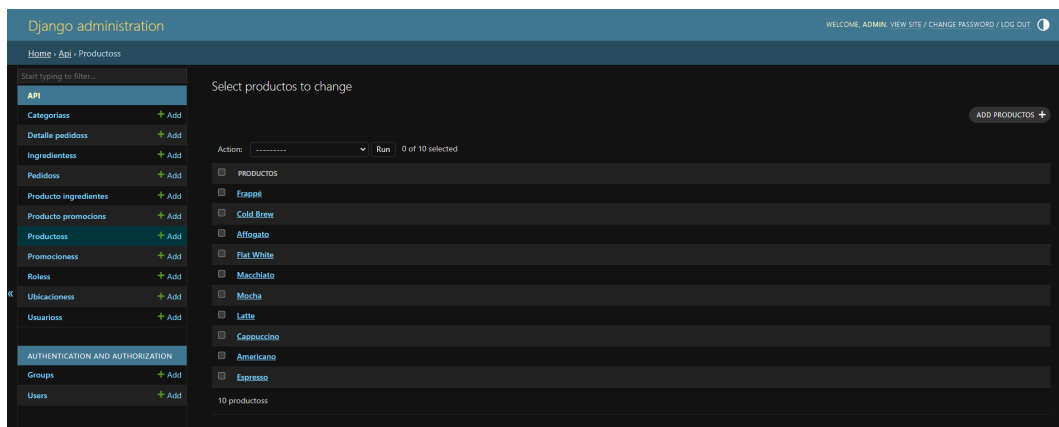


Figura 10: Interfaz de Django Admin mostrando las tablas del sistema conectadas a la base de datos PostgreSQL.

Los resultados evidencian que el modelo relacional diseñado inicialmente fue implementado correctamente en PostgreSQL y se encuentra plenamente integrado con el backend desarrollado en Django. Las pruebas realizadas confirman la consistencia del esquema, la integridad referencial y el acceso funcional a los datos desde la aplicación.

7. Autoevaluación

- **Fredy José Aragón Carpio (100 %)**
Responsable de la elaboración del modelo lógico DER, implementación del modelo físico en PostgreSQL, configuración de la base de datos en Supabase y redacción del informe técnico del laboratorio.
- **Gustavo Alonso Carrillo Villalta (100 %)**
Apoyo en la validación del modelo relacional, revisión de la estructura de tablas, claves primarias y foráneas, y verificación del cumplimiento de las convenciones de nomenclatura establecidas.
- **Diego Marcelo Arce Coaquira (100 %)**

Encargado de las pruebas de inserción y consulta de datos en PostgreSQL y Supabase, así como de la captura de evidencias y verificación del correcto funcionamiento de la base de datos.

■ **José Manuel Bravo Rojas (100 %)**

Responsable de la organización del repositorio en GitHub, elaboración del archivo README.md y revisión final de los entregables solicitados para la práctica.

La distribución porcentual refleja el nivel de participación y responsabilidades asumidas por cada integrante durante el desarrollo del proyecto.

8. Rúbrica de Calificación

A continuación se presenta la rúbrica de evaluación detallada para el presente laboratorio, considerando los criterios de modelado, implementación y documentación.

Ítem	Descripción	Puntaje
DER	Elaboración del modelo lógico DER.	4
Modelo físico	Implementación del modelo físico PostgreSQL.	8
Supabase	Implementación en Supabase.	0
Informe	El laboratorio tiene un README.md que detalla toda la práctica.	3
Prueba	Se tomaron en cuenta todas las consideraciones y recomendaciones, lo que evidencia un trabajo en equipo.	0
Total		15

Cuadro 1: Rúbrica de evaluación del laboratorio.

9. Referencias

- Supabase. *Sitio oficial*. Disponible en: <https://supabase.com/>
- PostgreSQL. *Documentación oficial*. Disponible en: <https://www.postgresql.org/docs/current/index.html>
- Sage Estimating. *Developing a Database Coding Scheme*. Disponible en: https://help-sageestimating.na.sage.com/en-us/23_1/Content/geninfo/developing_a_database_coding_scheme.htm
- Ovid. *Database Naming Standards*. Disponible en: <https://dev.to/ovid/database-naming-standards-2061>
- Stack Overflow. *What are some of your most useful database standards?*. Disponible en: <https://stackoverflow.com/questions/976185/what-are-some-of-your-most-useful-database-standards>
- Hwa. *Database Design and Code Standard*. Disponible en: <https://medium.com/@hwa610787/database-design-and-code-standard-89c0a4beee17>